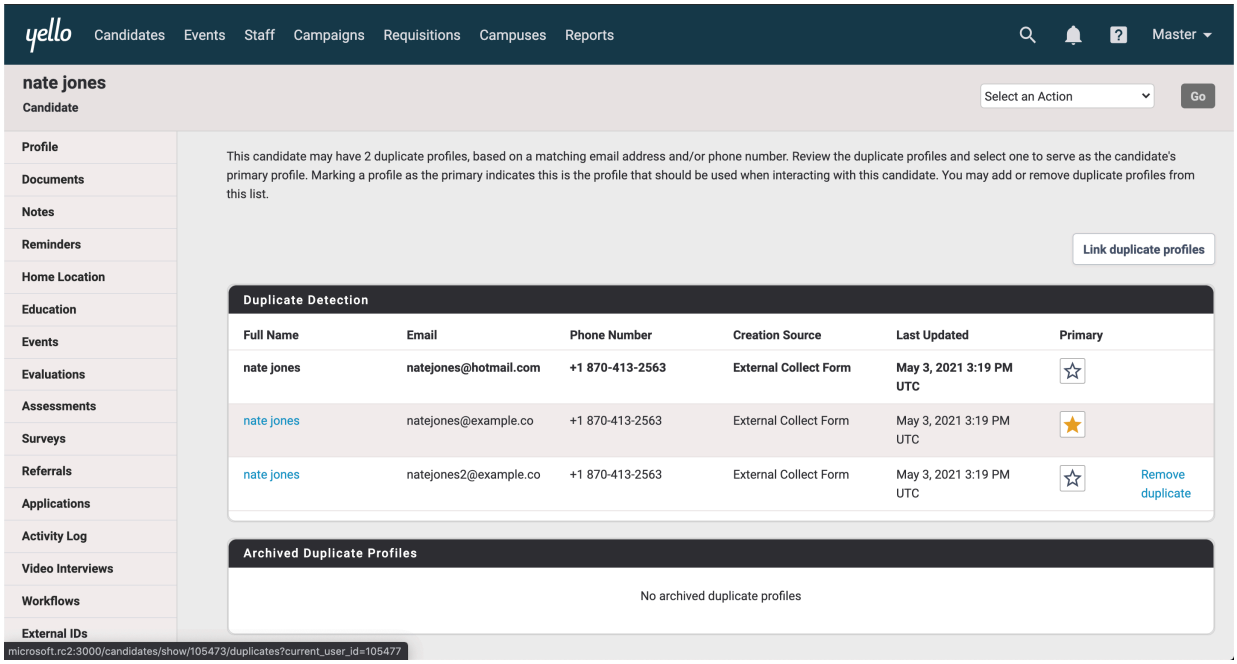
# Convention > Configuration

## The feature

Yello, a B2B enterprise platform for recruiting, has an extensive issue with duplicative data; specifically, candidate profile data.

Without going into too much detail, substantive data validations don't exist and recruiters have a tough time figuring out which candidate profiles are "current" and which are stale.

As a short term solution, my team was asked to build a feature that manages groupings of duplicate profiles, extending an existing solution; otherwise known as **duplicate detection**.



# The problem: messy views

When the project began, there was only:

1. Table of duplicate profiles
2. Instructional text
3. Sidebar

The above screenshot shows our goal:

1. First table of duplicate profiles
2. Second table of archived duplicate profiles
3. A few inline actions
4. A modal triggering button
5. Some instructional text
6. Sidebar

Seems straightforward enough. I'd go so far as to say that nothing about this view necessitates an unconventional or "innovative" approach; classic Rails conventions shine here.

However when locating the view in the application code, I had immense trouble.

Eventually I found the template file:

`app/views/users/candidates/profile/duplicates/show.html.erb`; what a mouthful!

The template file contained three partials:

1. `/users/partials/_user_profile_header.html.erb`
2. `/application/_aside.html.erb`
3. `/users/candidates/profile/duplicates/partials/_details.html.erb`

A few immediate questions:

- Why are `users` and `profile` used as organizational folders? The route is simply `/candidates/show/:id`; still unconventional, but worse still is now the controller action must explicitly reference the template file.
- Why is a `partials` folder used to organize partial files in some places but not others? More importantly, why does the folder even exist? Rails recognizes partials by pre-fixing file names with underscore (_) and conventionally references those files when calling `render(partial:)`.
- What else could `partials/_details.html.erb` contain other than a table?

From the get go, superfluous naming ensures the view is tough to find and difficult to remember. The view partials offer no clear data interface or concept of what it being rendered. If we are unable to rely on conventional Rails naming to locate or understand the view, it ensures all future developers must take extra time to learn this proprietary system.

# Crawling back to par

Truth be told, I had a grand vision to completely overhaul this show page, such that all routes, controller actions and view templates followed Rails guidelines.

However when you have bloated controllers and non-RESTful actions pointing to proprietary views, there is an immense amount of entanglement and only so much you can safely change.

And we had a 2-sprint time restriction (or 1 month).

Instead of refactoring everything at once, I focused on ensuring the

`/duplicates/show.html.erb` template was easier to understand and that all new work read conventionally.

To help guide the process, I asked the following questions:

1. What data does this view need?
2. What elements is this view rendering?
3. Is it clear which data partials are using?

Take the following line:

```
render partial: 'users/candidates/profile/duplicates/partials/details',
    locals: { presenter: @presenter}
```

This line is very confusing, both the name of the partial AND the presenter are obscuring the data and purpose of the element. It provides no clear data interface and forces us to learn specifics about `@presenter`, even though we know there's only one collection of data we need to render (duplicates).

After some data shuffling we landed here:

```
render partial: 'candidates/duplicates/table, locals: {
    user: @presenter.user,
    count: @duplicate_users.length,
    duplicates: @duplicate_users
}
```

Now the name is more consistent with where I would expect this view to be found: `candidates` view on the `duplicates` tab. Even better, the partial now represents a specific element or component: `table`, which we can reuse on this page for the archived list.

```
render partial: 'candidates/duplicates/table, locals: {
    user: @presenter.user,
    count: @archived_duplicate_users.length,
    duplicates: @archived_duplicate_users
}
```

Reusability is the logical next step because now I know exactly what data the partial expects, there's no more guessing.

Even though we only revised a single partial, we've cut short the pattern of copy pasting the same code everywhere. We've also left a footnote of how we think this view **should** be organized, i.e. `/candidates/:id/duplicates` or `/candidates/duplicates/_files.html.erb`.

Not only do these small actions add up in a large codebase, but we've improved the quality of life for all developers in the workspace. Getting the fundamentals right means establishing a clear

foundation with which to build more complex features. Otherwise without the bike shedding of common conventions, applications can quickly spiral into a spaghetti mess of entanglement.